

Tkinter: die Grundkonzepte

Tag 4

Vom Terminal zum klickbaren Programm

Was ist Tkinter?

Tkinter ist in Python **eingebaut**. Du brauchst nichts zu installieren.

Damit baust du Programme mit **Fenstern, Buttons und Eingabefeldern**.

```
import tkinter as tk
```

Ein Terminal-Programm läuft **von oben nach unten** und ist dann fertig.

Ein GUI-Programm **wartet** auf das, was der Mensch tut.

Das Hauptfenster

```
import tkinter as tk

window = tk.Tk()
window.title("Meine GUI")

window.mainloop()
```

- `tk.Tk()` erstellt das Fenster
- `title()` setzt die Titelzeile
- `mainloop()` hält das Fenster offen

Die Ereignisschleife (`mainloop()`)

`mainloop()` ist eine Schleife, die **ständig prüft**:

- Wurde geklickt?
- Wurde eine Taste gedrückt?
- Ist ein Timer abgelaufen?

Wenn ja, ruft sie die passende Funktion auf.

Ohne `mainloop()` ist das Programm **sofort wieder zu**.

Alles, was nach `mainloop()` im Code steht, läuft erst, wenn das Fenster geschlossen ist.

Widgets: die Bausteine

Alles Sichtbare in einem Fenster ist ein **Widget**.

Widget	Wofür
Label	Text anzeigen
Button	Klicken, löst eine Funktion aus
Entry	Text eingeben
Canvas	Zeichenfläche für Text, Bilder, Formen
Toplevel	zusätzliches Fenster

Widgets: erstellen und positionieren

Jedes Widget braucht **zwei Schritte**:

```
label = tk.Label(window, text="Hallo")    # 1. erstellen  
label.grid(row=0, column=0)             # 2. positionieren
```

Das **erste Argument** ist immer das Fenster, in dem das Widget liegt.

Ohne `grid()` ist das Widget **unsichtbar**.

Layout mit `grid()`

Das Fenster wird in **Zeilen und Spalten** aufgeteilt.

	column 0	column 1
row 0	A	B
row 1	C	D

```
a.grid(row=0, column=0)  
b.grid(row=0, column=1)
```

Die Zählung beginnt bei 0.

`grid()`: Abstand und Breite

```
label.grid(row=0, column=0, colspan=2, pady=10)
```

Option	Bedeutung
<code>colspan=2</code>	Widget geht über 2 Spalten
<code>padx</code> / <code>pady</code>	Abstand links/rechts bzw. oben/unten

Zwei Widgets in derselben Zelle liegen **übereinander**.

Button und **command**

Ein Button ruft beim Klick eine **Funktion** auf.

```
def hallo():  
    print("Hallo!")  
  
button = tk.Button(window, text="Klick", command=hallo)
```

`command=hallo` ✓ die Funktion wird **übergeben**, Tkinter ruft sie beim Klick auf.

`command=hallo()` ✗ die Funktion wird **sofort ausgeführt**, beim Start.

Widgets nachträglich ändern: `config()`

```
label.config(text="Neuer Text")  
label.config(fg="red")  
button.config(state="disabled")    # sperren  
button.config(state="normal")     # wieder freigeben
```

Eine Variable zu ändern ändert **nicht** automatisch die Anzeige.

Du musst das Widget **selbst aktualisieren**.

Eingaben lesen: **Entry**

```
eingabe = tk.Entry(window, width=20)
eingabe.grid(row=0, column=0)

name = eingabe.get()           # Inhalt lesen
name = eingabe.get().strip()   # Leerzeichen am Rand entfernen
eingabe.delete(0, tk.END)      # Feld leeren
```

`get()` liefert immer einen **String**.

Zustand speichern: `global`

Ein Klick ruft eine Funktion auf, danach ist sie **wieder vorbei**.
Was sich merken soll, steht in einer Variable **außerhalb**.

```
zaehler = 0

def erhoehen():
    global zaehler
    zaehler += 1
    label.config(text=str(zaehler))
```

Ohne `global` legt Python in der Funktion eine **neue, lokale** Variable an.

Das Grundmuster einer GUI

Klick → Funktion → Zustand ändern → Anzeige aktualisieren

```
def erhoehen():  
    global zaehler  
    zaehler += 1  
    label.config(text=str(zaehler))  
# Zustand ändern  
# Anzeige aktualisieren
```

Fast jede GUI-Funktion heute folgt diesem Ablauf.

Ein zweites Fenster: `Toplevel`

```
top = tk.Toplevel(window)
top.title("Zweites Fenster")

label = tk.Label(top, text="Hallo")
label.grid(row=0, column=0)
```

Widgets im zweiten Fenster bekommen `top` als erstes Argument, **nicht** `window`.

`top.destroy()` schließt das Fenster wieder.

Zeitgesteuert: `after()`

```
window.after(2000, hallo)
```

Ruft `hallo` nach 2000 Millisekunden auf (= 2 Sekunden).

Die GUI bleibt dabei bedienbar. `time.sleep()` würde das Fenster einfrieren.

Reihenfolge: Millisekunden, Funktion, Argument

```
window.after(1000, countdown, sekunden - 1)
```

Ein Countdown aus `after()`

Die Funktion ruft sich nach einer Sekunde selbst wieder auf.

```
def countdown(sekunden):  
    if sekunden == 0:  
        return # Ende  
  
    label.config(text=str(sekunden))  
    window.after(1000, countdown, sekunden - 1)
```

Die **Ende-Bedingung** muss vor dem nächsten `after()` stehen, sonst zählt er ins Negative.

Timer merken und abbrechen

`after()` gibt eine **ID** zurück. Mit ihr kann man den Timer abbrechen.

```
timer = None                                # es läuft keiner

timer = window.after(1000, countdown, 9)    # ID merken
window.after_cancel(timer)                  # abbrechen
```

```
if timer is not None:                       # es läuft schon einer
    return
```

In Funktionen, die `timer` ändern, braucht es `global timer`.

Minuten und Sekunden

```
minuten = 125 // 60      # 2    Ganzzahl-Division
rest     = 125 % 60      # 5    Rest

f"{minuten:02d}:{rest:02d}"    # "02:05"
```

`:02d` bedeutet: **zweistellig**, mit führender Null.

Canvas : die Zeichenfläche

```
canvas = tk.Canvas(window, width=240, height=150)
canvas.grid(row=0, column=0)

zahl_text = canvas.create_text(120, 75, text="00:10", fill="red")
```

- Position mit **x, y** in Pixeln (von links oben)
- `create_text()` gibt eine **ID** zurück, die du dir merkst
- Die Farbe heißt hier **fill**, nicht **fg**

Bilder auf dem Canvas

```
from pathlib import Path
ORDNER = Path(__file__).parent

tomato_img = tk.PhotoImage(file=ORDNER / "bilder" / "tomato.png")
canvas.create_image(100, 112, image=tomato_img)
```

- `Path(__file__).parent` ist der Ordner **neben der Datei**
- Das Bild muss in einer **Variable** bleiben, sonst verschwindet es

Daten sammeln: Strings aufbauen

```
checkmark = ""  
  
checkmark += "✓"  
checkmark += "✓"           # jetzt "✓✓"  
  
label_check.config(text=checkmark)
```

Die Variable allein ändert das Label **nicht**. Erst `config()` zeigt es an.

Lambda: Werte an command übergeben

```
command=naechste_phase(top)          # ✗ läuft sofort  
command=lambda: naechste_phase(top)  # ✓ läuft beim Klick
```

lambda: packt den Aufruf in eine kleine Funktion, die **erst beim Klick** ausgeführt wird.

Nötig, wenn die Funktion **einen Wert** bekommen soll.

Typische Fehler

Symptom	Ursache
Button tut nichts	<code>command=funktion()</code> mit Klammern
Widget unsichtbar	<code>grid()</code> vergessen
<code>Entry</code> liefert <code>None</code>	<code>Entry(...).grid(...)</code> in einer Zeile
Reset bricht nichts ab	<code>global timer</code> fehlt
Zählt ins Negative	Ende-Prüfung nach dem <code>after()</code>
Bild fehlt	Bild nicht in einer Variable gespeichert
Kein Fenster (Mac)	<code>Cmd + Tab</code> , <code>python3</code> verwenden

Zusammenfassung

Konzept	Merksatz
<code>mainloop()</code>	wartet auf Ereignisse
Widget	erst erstellen, dann <code>grid()</code>
<code>command</code>	Funktion ohne Klammern
<code>config()</code>	ändert ein Widget nachträglich
<code>global</code>	Variable außerhalb ändern
<code>after()</code>	später ausführen, ohne einzufrieren
<code>after_cancel()</code>	geplanten Aufruf abbrechen
Canvas	Elemente über ihre ID ändern

Wo brauchst du was?

Projekt	Konzepte
1 Begrüßung + Klick-Zähler	Fenster, Widgets, <code>grid</code> , <code>command</code> , <code>config</code> , <code>Entry</code> , <code>global</code> , <code>Toplevel</code>
2 Begrüßung + Countdown	<code>after</code> , <code>after_cancel</code> , MM:SS, <code>Canvas</code>
3 Pomodoro-Timer	Phasen, Häkchen, Bild, <code>lambda</code>